Applied Soft Computing Journal 174 (2025) 112973

Contents lists available at ScienceDirect

Applied Soft Computing

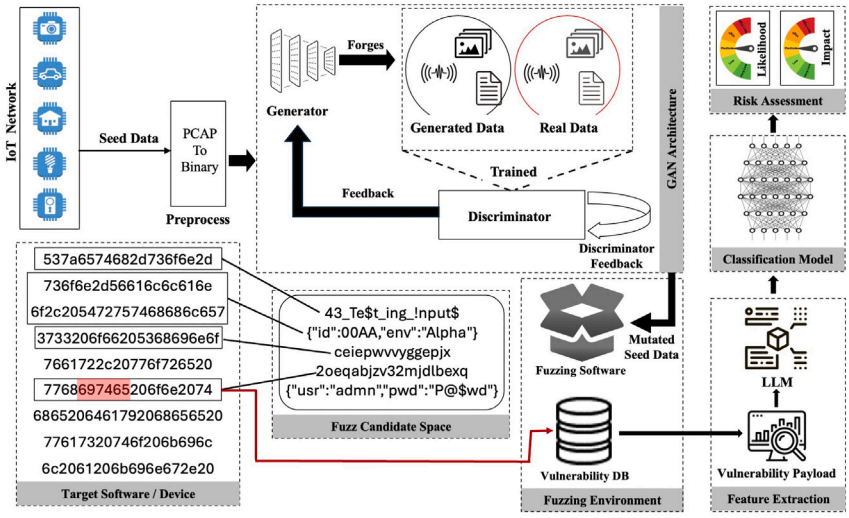
journal homepage: www.elsevier.com/locate/asoc

Generative fuzzer-driven vulnerability detection in the Internet of Things networks

Mohammed Tanvir Masud *, Nickolaos Koroniotis, Marwa Keshk, Benjamin Turnbull, Shabnam Kasra Kermanshahi, Nour Moustafa *

University of New South Wales, Canberra, Australia

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
Generative Adversarial Networks (GAN)
Fuzzing for vulnerability detection
Risk assessment
Deep learning-based fuzzing
Large language Model

ABSTRACT

The Internet of Things (IoT) paradigm has displayed tremendous growth in recent years, driving innovations such as Industry 4.0 and the creation of smart environments that enhance efficiency and asset management and enable intelligent decision-making. However, these benefits come with considerable cybersecurity risks due to inherent vulnerabilities within IoT ecosystems. Introducing potentially vulnerable IoT devices into secure environments, like smart airports, introduces new attack surfaces and vectors for exploitation. Identifying such vulnerabilities is challenging, and while traditional methods like penetration testing and vulnerability identification offer solutions, they often fall short due to IoT's unique data diversity, hardware constraints, and complexity. We propose an intelligent mutation-based fuzzer for IoT vulnerability detection in networks to address these limitations, demonstrated through a smart airport case study. This method leverages Generative Adversarial Network (GAN)-based mutation, utilizing legitimate network communications (i.e., payloads) to produce fuzzed payloads that expose vulnerabilities. Additionally, we incorporate a large language model (LLM)-based risk assessment framework to evaluate the likelihood and impact of identified vulnerabilities, which is crucial for effectively prioritizing threats in interconnected IoT environments. This dual approach of vulnerability detection and LLM-driven risk assessment provides comprehensive insights into IoT security,

* Corresponding authors.
E-mail addresses: mohammed_tanvir.masud@unsw.edu.au (M.T. Masud), nour.moustafa@unsw.edu.au (N. Moustafa).

https://doi.org/10.1016/j.asoc.2025.112973
Received 31 October 2024; Received in revised form 17 February 2025; Accepted 28 February 2025
Available online 10 March 2025

enabling prioritized response actions. Experiments conducted in the UNSW Canberra IoT testbed confirm that our approach outperforms conventional vulnerability identification methods, offering a scalable solution for effective vulnerability detection and risk prioritization in complex IoT networks.

1. Introduction

Contemporary innovations and rapid technological advancements have allowed the development of cheap, capable hardware and the efficient use of constrained power supplies, resulting in the emergence of a new domain known as the Internet of Things (IoT) [1,2]. IoT has gradually become an integral part of a number of industries and organizations and underpins modern residential, commercial and industrial areas. The IoT is comprised of sensors and actuators and other devices that have been outfitted with constrained processing, memory, network and power resources, interconnected into ubiquitous deployed networks that are capable of exchanging data, monitoring environmental conditions and adjusting their functionality according to user-provided or environment-derived stimuli. With an estimated 7 billion smart devices deployed in 2020 and projections predicting a rise above 38 billion by 2025, the importance and impact of the IoT in everyday life has become evident [3].

Typical IoT deployments combine devices, such as temperature, proximity and light sensors, with actuators such as air-conditioners, water/fuel pumps, ovens and refrigeration units, linking their outputs and internal states and enabling more complex and intuitive services that do not require direct user interaction [4,5]. This modular nature of the IoT, which enables the construction of multi-layered services, results in a collection of benefits that the IoT provides to its users, that have been realized by organizations and companies, including augmentations to existing business processes, automation, improved efficiency and flexibility and lower management costs [5]. These benefits have led to the genesis of a new domain, an extension of the IoT and a generalization of the Industry 4.0 paradigm, where environmental feedback is harvested from devices and consistently employed for intelligent decision-making, known as Smart Environments.

Smart environments integrate IoT devices to promote automation and efficiency, distinguishing them from traditional settings [5,6]. Examples include smart homes that improve residents' quality of life, smart agriculture that monitors crop growth conditions, and smart airports where IoT-enabled services optimize resource utilization and enhance passenger experiences [7,8]. This paper focuses on a smart airport as a case study, expanding on our previous work [5,7].

Despite the numerous advantages of IoT, significant security challenges persist. Cyber threats targeting IoT devices can exploit vulnerabilities to launch attacks such as Denial of Service (DoS), data exfiltration, and even large-scale botnet formation [9,10]. Additionally, the resource-constrained and always-active nature of IoT devices introduces unique threats like device bricking and resource depletion attacks [10]. In critical environments like smart airports, cyberattacks can cause widespread disruptions, with implications for public safety and national security [5]. Thus, effective cybersecurity solutions capable of detecting suspicious interactions and identifying vulnerable IoT devices in real-time or during downtime are crucial to maintaining the viability and security of IoT-driven smart environments [11].

Research Motivation- Existing solutions in research and industry often rely on either physically accessing IoT devices and assessing its integrity, often requiring invasive actions to be taken to gain access to its firmware, analysing the source code of mobile applications that control these devices. In contrast, others rely on the use of expert-derived rules [12–14]. Although such strategies can prove effective in detecting cyberattacks in smart environments, it may not be practical to inspect individual sensors and actuators in a smart environment, as due to their diminutive size and ubiquitous nature, it can be challenging to locate them and time-consuming to physically access them one-by-one. Furthermore, while monitoring the IoT network can be effective for the detection of attacks, this implies a post-mortem approach, where a device has already been compromised and actively affected the integrity of the entire smart environment network, for example by exfiltrating sensitive data, manipulating sensor readings or allowing lateral movement to an otherwise secured network.

The existing rule-based tools are the industry standard and have proven effectiveness for detecting known attacks, but they remain static and need expert maintenance in the form of new signatures and updates [15]. To overcome these limitations, fuzzing has been employed in network-based vulnerability detection by generating or mutating the actual data into pseudo-random inputs/probes that aim to unearth vulnerabilities within the target software. Fuzzing has been successfully used in other areas; however, direct application to IoT is faced with challenges. Most existing fuzzers fail to modify payloads intelligently, while no approach has been proposed yet that combines fuzzing and extensiveness of risk assessment, which allows prioritization necessary for response in smart environments [16–18].

To close this gap, we present the CNN Generative Adversarial Network-based Mutation Network Fuzzer (DCGAN-MNFuzz), a smart environment-enabled intelligent fuzzer tailored to IoT devices. By using a Generative Adversarial Network (GAN) architecture to intelligently generate mutated payloads based on real captures, DCGAN-MNFuzz can effectively test the target vulnerability by employing a diverse range of mutated network packets obtained from a diverse range of network-enabled devices with no reliance on companion applications. Developed for complex scenarios such as smart airports, this GAN-based model generates diverse, realistic network interactions to evaluate the robustness of IoT devices.

In addition, DCGAN-MNFuzz is combined with a novel LLM-based risk assessment framework that categorizes vulnerabilities into a range of criticalities according to detected risk levels, dependent on impact and likelihood of compromise. This ability to prioritize threats happens in real-time, which greatly mitigates the currently existing demand for up-to-the-second risk assessment within a dynamic, hyper-connected IoT system. While conventional methods identify vulnerabilities, they often lack the capacity to assess their severity in context. By combining fuzzing and dynamic risk assessment, DCGAN-MNFuzz ensures that security teams can focus on the most critical threats, significantly improving response times and the overall security posture of IoT networks.

Thus, this dual approach of GAN-driven fuzzing for smart mutation and LLM-based risk assessment for vulnerability ranking forms an end-to-end framework to secure IoT settings. In our experiments in the UNSW Canberra IoT testbed, we show how this combined framework improves detection and accuracy and enables scalable risk assessment, resulting in a substantial enhancement to IoT security solutions.

2. Background and related work

In this section, we provide a comprehensive review of existing research literature related to the domains of fuzzing, generative adversarial networks (GAN), and large language models (LLMs). In the discussion, the three domains are initially addressed separately, and then multidisciplinary research that leveraged the strengths of deep learning, either trained in GAN or a supervised configuration, to achieve deep learning-based automated fuzzing for the detection of both known and zero-day vulnerabilities in both mundane and smart environment networks. Finally, we examine recent uses of LLMs within fuzzing frameworks to evaluate vulnerabilities by assessing likelihood and impact, allowing for more targeted and prioritized security responses within IoT ecosystems.

2.1. Fuzzing

Originally, proposed in 1990 as a tool for evaluating the integrity of UNIX systems, fuzzing has been established as a reliable technique for identifying vulnerabilities in software, hardware, and network systems [19]. Programs designed to perform fuzzing scans, called fuzzers, rely on generating pseudorandom data sequences that resemble legitimate input for the software being tested. This generated input is fed to the targeted program that is being evaluated and processed, and the output is then extracted and analysed by the fuzzer [20]. A crash triggered as a result of employing a fuzzer's pseudorandom input or forcing the targeted system to behave unexpectedly is an indication of a bug, vulnerability, or uncaught exception. Such crashes can be leveraged to seek temporary mitigation tactics until the exact cause of the crash is detected in the code and patched. Fuzz testing can be divided into three kinds based on the knowledge of the target system: white, grey, and black box testing. White box testing utilizes full source code knowledge to improve input generation and code exploration. Black box testing ignores the internal structure and, instead, focuses on rapidly scanning the system. Grey box testing adopts both by starting like a black box and using feedback to refine input generation for better coverage [21].

Grey-box fuzzing is a lightweight software instrumentation approach that generates test inputs automatically. Since being made widely known by open-source tools like AFL [22], it has attracted a lot of attention from academic researchers, resulting in several additions and developments in the area [21,23,24]. Grey-box fuzzing works in a generate-and-execute loop of test inputs on the target application. It begins with initial inputs by constructing a seed queue and instrumenting the target for coverage tracing. The steps in the loop are (1) Seed Selection - selecting the most promising seeds; (2) Resource Optimization - optimizing resources; (3) Input Mutation - mutating the seeds via bit flips or splicing; and finally, (4) Test Execution - running the mutated inputs on the target. Found vulnerabilities are reported, the seed queue is updated, and the cycle continues, improving input coverage.

2.2. Generative adversarial networks

Introduced by Goodfellow et al. in 2014, Generative Adversarial Networks (GANs) consist of two competing neural networks: a generator and a discriminator. The generator creates fake data, while the discriminator distinguishes between real and fake samples. Through adversarial training, both networks improve iteratively. The generator's output must match the discriminator's input shape. The training combines supervised learning for the discriminator (using labelled data) and unsupervised learning for the generator (using forged data). This zero-sum game continues for a predetermined number of iterations, gradually enhancing both models' performance [25].

In neural network-based GANs, the learning process resembles backpropagation, as it would occur normally but adapted to the purposes of the models. On the one hand, the generator aims to produce increasingly realistic fake data to minimize the discriminator's accuracy. On the other hand, the discriminator itself aims to correctly identify what samples are real and what is not by maximizing its classification accuracy. Let z be the pseudo-random vector input to the generator, $G(z)$ be the output of the generator, which has the same dimensions as original data X, $D(X)$ and $D(G(z))$ represent the discriminator's output for original and generated data, respectively. Eq. (1) represents the GAN min–max loss function. Here, 'min' corresponds to the generator's goal of fooling the discriminator into believing the generated fake data is real. At the same time 'max' is representative of the discriminator trying to correctly classify real versus fake data.

In Eqs. (2) and (3), L_D and L_G represent the generator and discriminator losses, respectively. The generator minimizes L_G to produce realistic samples, while the discriminator L_D maximizes to correctly classify real and fake inputs. Eq. (3) provides the individual cost functions for the generator, which differs from the GAN loss, as the generator cannot affect its first component $E_X[log(D(X))]$. Note that the discriminator maximizes Eq. (2), as this equates to maximizing $log(D(X))$ and thus, because $log(x)$ is a positive monotonic function, maximizing $D(X)$ equal to '1' which corresponds to real data being classified as real, as $D(X) \in [0, 1]$, while maximizing $log(1 - D(G(z)))$ leads to maximizing $1 - D(G(z))$, for which $D(G(z))$ needs to be minimized to '0', to correctly classify fake data as fake. On the other hand, the generator seeks to minimize the same equation; however, its input is only relevant for the second part of Eq. (1), thus leading to Eq. (3), where it seeks to minimize $log(1 - D(G(z)))$ thus maximize $D(G(z))$ to '1', which corresponds to miss-classifying forged data as real. Fig. 1 presents an abstract view of a GAN configuration during training.

$$L_{GAN} = min_G max_D[E_X[log(D(X))] + E_Z[log(1 - D(G(z)))]] \quad (1)$$

$$L_D = max_D[E_X[log(D(X))] + E_Z[log(1 - D(G(z)))]] \quad (2)$$

$$L_G = min_E E_z[log(1 - D(G(z)))] \quad (3)$$

As a result of the versatility of GANs, supporting multiple machine learning types and not restricted to specific data types, their applications are numerous, and span multiple disciplines, including science, video games, health and cybersecurity [26–29]. Motivated by an investigation of the effectiveness of existing methods for the generation of malware image samples and stating the need for realistic and fully labelled datasets, Singh et al. [30] proposed MIGAN, an augmented Auxiliary Classifier Generative Adversarial Network-based (AC-GAN) method. The work initially processes malware binaries, converting them into fixed-dimension grey-scale images, and leverages the AC-GAN to automatically assign the appropriate malware category, thus providing automated labelling. The proposed method can be utilized as a way of enhancing existing malware datasets or producing novel ones that can be leveraged for the design of deep learning-based solutions. Experiments showed promising performance for the AC-GAN, gauged by Structural Similarity (SSIM) scores and Frechet Inception Distance, outperforming the previously established AC-GAN in sample quality and consistency.

Focusing on generating fuzz test cases, Feng et al. [31] introduced Sizzler, a fuzzing framework for vulnerability detection in Programmable Logic Controllers (PLCs), which utilizes SeqGAN to analyse ladder diagrams and generate various test cases that focus on fuzz. Using QEMU emulation, Sizzler discovered several vulnerabilities such as race conditions and integer overflows, and is scalable across different PLC architectures. Another example is the GAN-based ICSP (Integrated Circuit Busting Catalyst Subsystem) fuzzing framework developed by Yang et al. [32] which uses WGAN (Wasserstein Generative Intelligent Network) and VAE (Variation Autoencoder) to process real network traffic, to generate various combinations of protocols while achieving a 91% test case recognition rate that can identify vulnerabilities such as integer overflows. The creation of SGANFuzz, a GAN-based fuzzing technique for MQTT protocol sequences, was developed by Wei et al. [33] who discovered six vulnerabilities, including three CVEs, that were more effective than conventional fusing devices in terms of acceptance rates and network responses. The GAN-based approach, which includes an LSTM generator and CNN discriminator, was utilized by Hu et al. [34] to develop GANFuzz, an industrial network protocol fuzzer, demonstrating its effectiveness in identifying Modbus-TCP vulnerabilities. In addition, Ye et al. [35] introduced RapidFuzz, an extension of AFL that enhances mutation by incorporating more test seed files into a GAN-based fuzzer.

Although the presented work leverages GANs to great effect, some solutions focus on enhancing and complementing existing tools. In contrast, others rely on extracting protocol vocabularies and leveraging them in generative-based fuzzing for either software or network protocols. In our work, we propose an intelligent mutation-based fuzzer

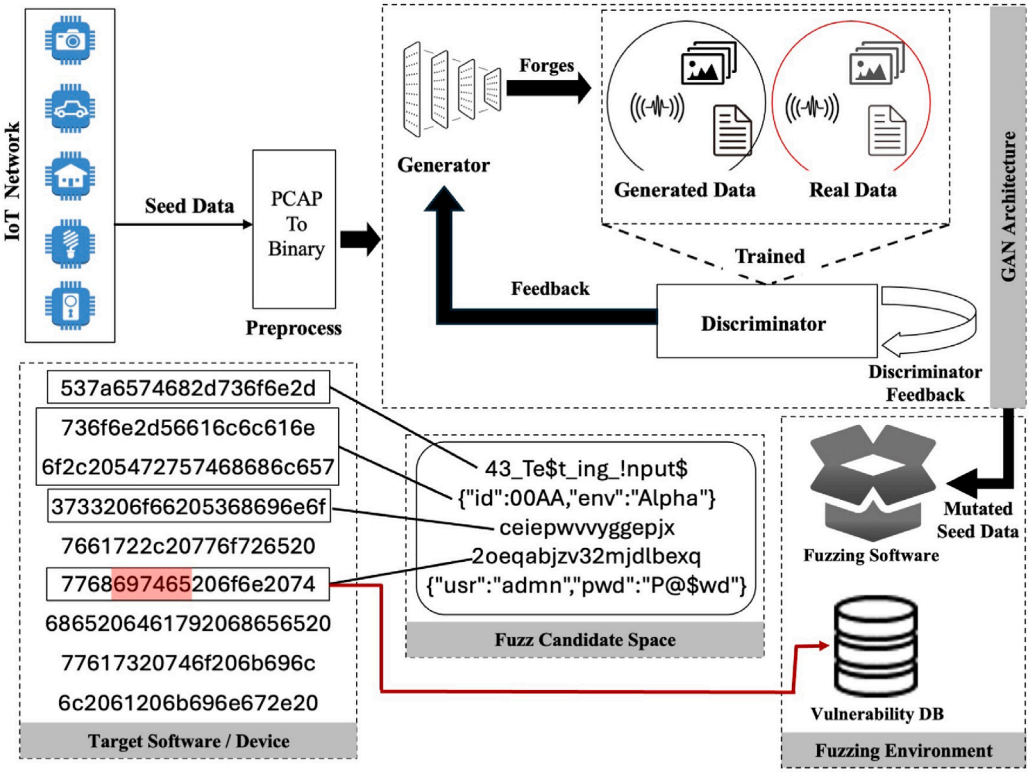


Fig. 1. Abstract representation of mutation-based fuzzing process.

in the form of a CNN GAN generator that is trained on legitimate and illegitimate IoT network communications and then mutates them, ensuring that the produced fuzz test cases will be of correct form while eliminating the need for a dedicated per-protocol vocabulary.

2.3. Large Language Models (LLMs)

LLM-based models have many parameters and are known for their excellent learning skills, allowing them to perform very well on Natural Language tasks like text completion and conversation [36]. Trained on massive corpora and models, and performed some tasks without additional training or manual programming. To interact with an LLM, a human user must submit a prompt, which comprises a specific command or query articulated in textual format. The LLM produces replies contingent upon the input supplied via the prompt [37].

Given that LLM capabilities have far-reaching effects on network protocols, LLM advances heavily impact the protocol space. Wang et al. [38] presented LLMIF, a fuzzing framework that enhances LLMs with protocol specifications to derive message formats, and dependencies and facilitate seed modification. This methodology enhanced message coverage by 55.2% and identified 11 vulnerabilities, comprising eight zero-day exploits, in actual Zigbee devices. Building on this, Meng et al. [39] presented an LLM-guided protocol fuzzing, a fuzzing framework that employs LLMs to capture machine-readable grammars from Request for Comments (RFCs) and drive message mutation. Generating input messages using this method achieves better state and code coverage than previous tools such as AFLNET [40] and NSFUZZ [41], and has discovered nine new vulnerabilities, demonstrating the potential of LLMs to improve fuzzing protocols. In a similar vein, Ma et al. [42] introduced mGPTFuzz, a fuzzing framework for Matter IoT devices that transforms human-readable Matter specifications into machine-readable format using LLMs, thus providing a basis for stateful fuzzing. 147 bugs were found in all, with 61 of them zero-day vulnerabilities (three assigned CVEs).

Despite prior research demonstrating significant success in protocol fuzzing by dynamic seed mutations utilizing LLMs, they have yet to concentrate on providing a detailed description of the identified vulnerabilities or elucidate their significance. Moreover, these studies have yet to tackle vulnerability assessment, a vital component of cybersecurity. We depend on cybersecurity professionals' physical observation and manual analysis to evaluate and articulate the risks linked to identified vulnerabilities [43], a domain where LLMs can significantly contribute to addressing a crucial gap in the existing cybersecurity framework.

3. Methodology

In the following section, we explain our proposed method, Intelligent Network IoT Fuzzer, in detail, named DCGAN-MNFuzz. This section briefly describes the fuzzer itself (the neural network architectures and hyperparameters used to drive its core functionality) and a high-level development strategy behind it to explain these choices. We further talk about integrating the LLM-based risk assessment framework, which analyses and prioritizes vulnerabilities discovered by fuzzer. Then, we present the data sources and pre-processing techniques as well as DCGAN-MNFuzz high-level overview in terms of a cycle alluding to the fuzzer's full functionality expressed within a dynamic IoT network landscape.

3.1. Fuzzing strategy: Mutation

In the context of fuzzing, extensive research has produced two main strategies with regard to the selection of candidate fuzz probes that are forwarded to the target of the fuzz testing (be it device(s) or software) to evaluate its stability and identify potential vulnerabilities, those being: generation and mutation [20]. Generation-based fuzzing requires considerable preparations to define the multiple states and parameters of communication protocols, software and/or hardware devices. Candidate fuzz probes are generated either randomly, thus falling

into the category of unsophisticated fuzzers, or from an elaborate model replicating the target device's input, its legitimate states and the permitted transitions between them. As such, generation-based fuzzers require extensive effort by experts to ensure such tools are kept up-to-date and can effectively interact with the latest versions of devices and protocols. We have elected to implement a mutation-based strategy to overcome such limitations present in generation-based fuzzing.

Unlike generation strategies, mutation-based fuzzing relies on legitimate messages stored in seed files that maintain a valid format that is recognized and accepted by the target devices, thus reducing the probability of rejection of the mutated messages by the fuzzing targets. In the context of network fuzzing, candidate probes are produced by mutating legitimate data that has been observed and captured in communications between the target and other devices in the network. These mutations rely on bit or byte-wise flipping, adding values and other such interactions, often guided by some feedback, such as code-coverage and vulnerability discovery effectiveness. To improve the effectiveness of mutation fuzzing tests, mutated data that has resulted in the discovery of a vulnerability (through device crash, restart or other unusual behaviour), is often marked for further mutation, before the fuzzer proceeds to the next candidate message found in the seed file. To avoid unnecessary rejections of fuzz probes, occasionally, fuzzers will seek to identify protocol-sensitive segments of messages and exclude them from the fuzzing process [44,45]. Unlike existing work in literature, we have selected to implement mutation fuzzing through Deep Learning.

3.2. Deep learning-enabled fuzzing

The approach that was selected for the implementation of the Intelligent Mutation-based fuzzer, as is presented in this paper, takes advantage of the automated and mostly unsupervised nature of Generative Adversarial Networks (GAN), where a pair of models, namely a **generator** and a **discriminator** compete in a 0-sum game, improving their performance by leveraging the discriminator's misclassification feedback [25]. Thus, the mutator of the mutation-based fuzzer, which is represented by the generator of the GAN, takes the form of a deep learning model composed of a stacked deep Convolutional Neural Network (CNN) and the dense layers of a Multi-Layered Perceptron (MLP). The goal of selecting GAN as the training configuration for a ML/DL model is to produce a model that, after a number of epochs have elapsed, is capable of generating realistic data of a particular type, such as images, sound, video and text. However, to do so, an initial sample dataset of the selected type needs to be constructed and curated (pre-processed) before it can be of any use to the GAN discriminator. For the purposes of this paper, the type of data that was collected was in the form of payloads, obtained from pcap files, extracted from the network interactions of the local IIoT devices of the SAir-IIoT testbed [7].

3.2.1. Data collection and pre-processing

As the motivation of the presented work was to produce an intelligent mutation-based fuzzer for smart environments comprised of IIoT sensors and actuators, for the underline DL models to be effective at detecting vulnerabilities over the network, they needed to be trained on realistic and representative data. To achieve this, a testbed developed in our previous work for this purpose, the SAir-IIoT [7], was selected as the experimentation environment, as it consists of 4 independent zones representing both internal and external environments of a smart airport, outfitted with a plethora of interconnected (I)IoT devices. Devices in each zone were configured to either post telemetry data to MQTT brokers or directly communicate with non-IoT devices, producing network interactions that were captured and stored in pcap files. Additionally, before training and launching the mutation fuzzer, automated penetration testing is performed, enabling the collection of abnormal activities in the network.

With permission from the network administrator and access to encryption keys, we can decrypt payloads, enabling unrestricted analysis of network interactions captured in pcap files. During pre-processing, several JSON files are constructed according to information obtained from pcap files, where flow identifiers for each flow captured in the pcap are stored (source and destination IP and port number, protocol), with the payloads stored in the form of conversations, in the form of a list that stored the payload as a text along with the direction of the message (source to destination and vice versa). These extracted conversations are then converted into binary format, and stored in a backend database, along with a class feature, that determines if the payload is part of a normal interaction (value of '0') or part of exploiting a vulnerability (value of '1'). To impose uniformity on the binary sample interactions, which is a requirement due to the CNN-MLP architecture of the neural networks that are utilized for the mutation process, padding in the form of '0' values is added at the beginning of each sample, to make them equal in length to the largest observed sequence. This process can be seen in algorithm 1.

3.2.2. Automated data labelling

When creating a dataset or preparing data for the construction of automated DL-based solutions, certain actions need to be performed involving pre-processing to transform the data so that it can be processed by the models. Although pre-processing is crucial for the effective training of DL and ML models, another crucial and non-trivial step that requires attention is labelling. Labelling is the process of assigning values to the class feature (dependent variable), correctly identifying an event that a record represents. Depending on the intended purpose of the collected dataset, the class feature can take many forms, such as binary for binary classification, numeric for multiclass classification and textual for multilabel classification. In terms of network flow traffic, a network flow that has been collected from a network while an SYN-flood denial of service (DoS) attack was taking place would be correctly classified as an attack, with secondary features indicating the type of attack as "DoS" and the subtype as "SYN-flood". Ensuring that all dataset records are correctly labelled is imperative, as DL models trained on mislabelled data will display a considerable error rate, either misclassifying benign instances as malicious (false positive) or failing to detect attacks (false negative). How labelling is carried out depends on the design and configuration of the data collection system, however often the process is either manual, with a domain expert reviewing data and assigning labels accordingly. In environment configurations, where the attacking and data collection machines are separated, labels are assigned based on temporal data values and flow identifier features, requiring a degree of synchronization in both machines.

The training, evaluation and operation of the intelligent mutation-based fuzzer relies on collected network interactions from both benign and malicious scenarios, acquired from the hybrid cybertwins smart environment network in the Sair-IIoT testbed. As such, it is vital to impose uniformity of data and accurate labelling to ensure that the fuzzing process can be carried out without false positives or false negatives that would result in normal devices identified as vulnerable and active vulnerabilities going undetected, respectively. Thus, as part of the network interactions collection process, we implemented an automated data labelling procedure that does not require using temporal data values for accurate labelling. The modules responsible for data collection and preliminary vulnerability test scanning are situated in the same machine, with the latter representing the malicious scenarios. They have been configured to coordinate for accurate automated labelling. First, the preliminary vulnerability testing module is engaged. It actively assesses every open port of every live host in the network against a list of well-established vulnerabilities, limiting the tests to relevant exploits according to the detected service of each port. If a vulnerability is deemed successful, then the data collection module is activated, recording the network interaction between the attacking machine and the target, the type of exploit that was utilized and the

Table 1
Architecture of the GAN generator for mutation-based fuzzing.

Hyperparameter	Value
Dense layer (First Layer)	2560 units (256 × 10)
Hidden layers	11 (10 Conv1DTranspose + 1 Flatten)
Conv1DTranspose details	(Filters, Kernel, Stride) → (128, 6, 2), (64, 5, 2), (32, 4, 2), (16, 3, 2), (8, 4, 2), (128, 6, 2), (64, 5, 2), (32, 4, 2), (16, 3, 2), (8, 4, 2)
Batch normalization	Yes (After each Conv1DTranspose)
Flatten layer	Flattens $(160, 16) \rightarrow (2560,)$
Activation (Hidden Layers)	ReLU (for all layers)
Activation (Output Layer)	Sigmoid
Batch size	65 (primary)
Epochs	4000
Optimizer	Adam
Loss function	Binary Cross-Entropy
Input latent dimension	$latent\_dim$ (derived dynamically)
Output shape	$(latent\_dim, 1)$

flow identifiers. Collected records, in binary format, are labelled and stored in a persistent medium for use during training/evaluation of the intelligent mutation-based fuzzer. At the same time, during operation they are utilized as input to produce mutated data.

3.2.3. Mutation DL architecture

Implementing a mutation-based fuzzer can be accomplished through conditional hard-coded and feedback-driven operations where different segments of the fuzz sample data are altered (bit flipping, addition, random data injection, etc.) to ensure effective code coverage. However, in this work, a DL-based method was selected for the mutation process, where the mutator takes the form of a generator model and is trained as part of a GAN on pre-processed network exchanges that have been collected from the SAir-IIoT environment.

- **Novel Generator for fuzzing mutations-based GAN:** The generator model of a GAN is tasked with converting a pseudorandom input into realistic data, improving its output with regards to quality after each epoch and by leveraging the discriminator misclassification error. In the context of the mutation-based fuzzer, we have designed the generator to consist of multiple layers arranged in a sequential model. First, dense neuron layers with nonlinear activation functions are leveraged as the input layer that receives the padded binary sample and produces a binary mutated sample (re-shaped to the correct dimensions to form a 1-D array). To ensure the dimensions of the output are correct, an upsampling process is required, and to that effect, we utilize transpose convolutional layers, with the output having a height and depth of 1 and a width equal to the maximum recorded binary payload length (recall that the number increased in convolutional layers, and decreases with transpose convolutional).
  To model the mutation process, input and output are defined to have the same dimensions $(1, 1, max\_binary\_length)$, and thus *Reshape* layers are added to the architecture of the generator. The generator leverages the following layers: convolution, max pooling, 1-D convolution transpose, dense, batch normalization, flatten, reshape, ReLU activation, and Sigmoid activation. The specific design choices and hyperparameters of this generator model are detailed in Table 1, outlining the key layers, activation functions, and optimization settings.
  Additionally, the generator can handle encrypted data, as it leverages Algorithm 1 in the pre-processing stage to transform collected network interactions and make them accessible by the DL models of the fuzzer. This algorithm performs two main tasks: binary conversion and padding. Converting collected network interactions into binary format enables ML/DL models to receive them as input. Ensuring a uniform length by applying common padding equal to the maximum observed character length of all collected network interactions is required, as non-sequential DL models require a fixed input size. The novel generator for fuzzing mutations-based GAN leverages collected network interactions as input, producing mutated outputs that the GAN discriminator, along with real network interactions, is tasked with classifying. The discriminator error trains the novel generator, resulting in a model that can produce realistic mutations.
- **GAN Discriminator:** A discriminator is required to train the mutator (GAN Generator). To that effect, we defined a discriminator model consisting of convolutional layers for downsampling (the dimensions of the output of a convolutional layer are given in Eq. (4)), dropout layers to avoid overfitting and a flattened layer to connect the final convolutional layer to a series of dense layers, to enable binary classification. In this equation, i_{out} represents the output size of the feature map, i_{in} is the input size, p is the padding applied to maintain spatial dimensions, f is the filter size, and s is the stride length. This equation ensures that the downsampling process in the discriminator preserves structural integrity while effectively reducing dimensionality for better feature extraction.
  The task of the discriminator is to detect the forgeries produced by the mutator and thus provide it with the necessary feedback that can be used to improve its output quality. A sigmoid activation function (5) is used in the final layer, and the selected loss function is binary cross-entropy (6). The discriminator of the GAN configuration functions as a filtering method for improving the quality of the output of the mutator. To ensure that the performance of the intelligent fuzzer is at an appropriate level for real-world deployment, a cut-off threshold of 99% is set for the discriminator of the GAN configuration. Once the discriminator reaches an accuracy of 0.99 (99%), after which the training of the mutator is terminated.

$$i_{out} = 1 + \frac{i_{in} + 2p - f}{s} \tag{4}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^{x}}{1 + e^{x}} \tag{5}$$

$$L(x) = -\frac{1}{N}\sum_{i=0}^{N} y_i log(\hat{y}_i) + (1 - y_i) log(1 - \hat{y}_i) \tag{6}$$

$$R(x) = \max(0, z_i) = \begin{cases} z_i, if & z_i > 0 \\ 0, if & z_i \leq 0 \end{cases} \tag{7}$$

  where i_{out} and i_{in} are the output and input dimensions respectively, p indicates the added padding ($2p$ as padding of equal length is added on either side of the array), f is the dimension of the filter kernel that is applied to the input array and s is the number of cells the kernel moves before each calculation.
- **Vulnerability discriminator:** In addition, the fuzzer maintains a separate vulnerability discriminator, that is, a classifier model that is tasked with determining if a payload, in pre-processed, padded and binary format, is indicative of a vulnerability being

exploited or part of normal network interactions between the IIoT devices. This discriminator is crucial to the process of vulnerability detection, as based on this model's output, new vulnerabilities can be found.

Algorithm 1 Encoding of IoT network interactions to binary format.

```
Input: max_len, input_msg
Output: final_bin
max_len = get_max_payload_length()
input_msg = load_msg()
input_msg_hex = convert_num(input_msg, base=16)
input_bin=convert_bin(input_msg_hex)
remainder=max_len-length(input_bin)
final_bin=input_bin
if remainder > 0 then
    final_bin=remainder*"0"+final_bin
end if
final_bin=reshape_list(final_bin,dim=(max_len,1))
return final_bin
```

Algorithm 2 Preliminary multi-process vulnerability testing.

```
Input: List of live hosts ('host_list'), list of network exploits ('exploit_list')
Output: Stores successful exploitation details (host IP, port, exploit)

initiate_network_recording()
host_list = get_live_hosts()
exploit_list = local_network_exploits()
host_sublist = []
subprocess_list = []
for host in host_list do
    host_sublist.add(host)
    if length(host_sublist) == 2 then
        pid = new_Process(target = sprcs, args = (host_sublist))
        subprocess_list.add(pid)
        host_sublist = []
    end if
end for
for subP in subprocess_list do
    subP.wait()
end for
Function test(host, exploit)
cnct = connect(host.IP, host.Port)
ld_explt = locat(exploit)
result = send_exploit(cnct, ld_explt)
Function sprcs(host_sublist)
for host in host_sublist do
    for exploit in exploit_list do
        if test(host, exploit) is True then
            store(host.IP, host.Port, exploit)
        end if
    end for
end for
```

3.2.4. Fuzzing scenario

The functionality of the proposed intelligent mutation-based fuzzer is displayed in Fig. 2. In the 1st stage, the local network is scanned, and all live hosts and open ports are recorded in dedicated tables of the backend database to ensure that inactive IP addresses are ignored during the later stages of the fuzzing process. Concurrently, a separate module loads and stores all available network-based exploits through the Python-Metasploit interface in a separate table of the same database. In the next (2nd) stage, triplets of live IP addresses, open ports and exploits are constructed and forwarded to the penetration testing module, which facilitates the preliminary penetration testing, as displayed in algorithm 2. To optimize performance, we parallelized the vulnerability scanning process by manually assigning pairs of hosts to a single process. This greedy approach to parallelism improves efficiency, as a single-process scan required extensive time to complete. While diminishing returns may occur depending on the host's capabilities, this approach was selected as a proof of concept for automated penetration testing and vulnerability detection, where speed and efficiency were not the primary focus. Each network-based exploit is evaluated against the list of live IP-port pairs, and any successful exploits are recorded in the *foundVulnerabilities* table so that the fuzzing process can be

Algorithm 3 Preliminary vulnerability testing feedback strategy.

```
waitT = 5
totalWaitT = 0
while acquire_feedback() is None do
    sleep(waitT)
    totalWaitT = waitT
    waitT = waitT - 0.5 * waitT
    if waitT ≤ 0 then
        waitT = 1
    end if
    if totalWaitT ≥ 20 then
        break
    end if
end while
```

Algorithm 4 Training process of Fuzzer Mutator.

```
Input: Seed data
Output: Trained generator model
generator = load_generator_model()
discriminator = load_discriminator_model()
gan_model = combine(generator, discriminator)
batchS = get_batchSize()
batchS_H = batchS * 0.5
epochs = load_N_epochs()
i = 0
for i ≤ epochs do
    real_data = load_data("network", size = batchS_H)
    fake_data = generator.run(random_interactions(), size = batchS_H)
    discriminator.train([real_data, fake_data])
    realistic_forged_data = generator.run(random_interactions(), size = batchS)
    gan_model.train([realistic_forged_data], disable = discriminator)

    i += 1
end for
store_model(generator)
```

Algorithm 5 Training of Vulnerability Detection Discriminator.

```
detector = load_vuln_detector()
batch = get_batchSize()
epochs = load_N_epochs()
i = 0
X, Y = load_labelled_payloads()
X_bin = str_tp_bin(X)
x_trn, x_tst, y_trn, y_tst = train_test_split(X_bin, Y)
for i ≤ epochs do
    detector.train(x_trn, y_trn, batch)
end for
```

Algorithm 6 Automatic Labelling of Captured IIoT Network Payloads.

```
extract_payloads_from_PCAP_files()
f = open_file_path('./captures')
for file in f do
  if file.ends_With('.json') then
    convos = json.load(file)
    if file.type == 'Normal' then
      cls = 0
      tp = 'Normal'
      stp = 'Normal'
    else
      cls = 1
      tp = file.type
      stp = file.subtype
    end if
  else
    continue
  end if
  load_to_DB(convos, class = cls, type = tp, subtype = stp)
end for
```

Algorithm 7 Comprehensive Vulnerability Feature Extraction and Likelihood/Impact Assessment Process.

```
Input: Vulnerability_Payload, Protocol, Port
Output: Extracted features and likelihood/impact assessments
Load Vulnerability_Payload and preprocess the data
Extract Features:
  Construct prompt for feature extraction
  Run prompt through LLM using subprocess
  Capture raw response
if response is valid then
  return Parse response for features
else
  return Error indicating failure
end if
Assess Likelihood and Impact:
  Construct prompt for likelihood and impact assessment
  Run prompt through LLM using subprocess
  Capture raw response
if response is valid then
  return Parse response for likelihood and impact
else
  return Error indicating failure
end if
Initialize results list
for each payload in Vulnerability_Payload do
  if payload is valid then
    features ← Extract Features
    likelihood_impact ← Assess Likelihood and Impact
    results.append(merge(features, likelihood_impact))
  end if
end for
Return results
```

expedited by omitting already scanned ports with found vulnerabilities. During the host discovery and preliminary penetration testing stages, and based on user prompts, network interactions between the IoT devices and other network-enabled entities are initially recorded in pcap files.

The pcap files are then processed, with the payloads extracted and stored in the form of JSON files, each of which stores the two ends of a communication (source and destination IP addresses and port numbers), and a list of the exchanged payloads along with their direction (source to destination and vice-versa). These payloads are crucial to the fuzzing process, as they are leveraged for the training of both the mutator, which is responsible for altering existing valid payloads and the discriminator that differentiates between normal and abnormal (vulnerability-indicative) network responses that are received after the mutated payloads are dispatched. In the 3rd stage, the DL models that provide the core functionality to the fuzzer are trained and evaluated on data that has been collected during the other two stages. First, a deep MLP-based discriminator is trained that takes the form of a binary classifier and outputs the value '0' for normal interactions and '1' otherwise, that is responsible for detecting vulnerabilities based on the responses that the fuzzer receives to its mutated probes. Then, in the preliminary penetration test, by leveraging the captured network interactions, the mutator model is trained.

The mutator takes the form of the generator from a pair of deep neural networks with stacked deep and convolutional (and transpose convolutional) layers that were described in the previous subsection and are trained in a GAN configuration, where the error from the GAN discriminator is utilized by the GAN generator to periodically improve the quality of produced data. The process is identified as mutation-based fuzzing, as the input of the GAN discriminator is set to be the captured vulnerable-indicative interactions, thus resulting in the generator producing realistic mutated output. The previously captured network interactions are loaded and forwarded to a dedicated messaging module, where they are first converted into binary format (as per algorithm 1), mutated by the stored trained GAN generator, and converted back to text from binary, before it is sent to the remote host and a response is received. Any acquired responses are parsed through the discriminator (not to be confused with the GAN discriminator) and labelled as vulnerable-indicative or normal. For any detected vulnerabilities, the IP address, port number and port type of the target, along with the mutated payload, are stored in the *foundVulnerabilities* table of the backend database for later reference.

3.3. LLM driven features extraction and risk assessment

The underlying architecture of the LLM used for risk assessment is based on Ollama 3.1 running the Llama 3 model, a transformer-based deep learning model designed for contextual understanding and classification. The LLM is utilized for extracting 14 key features from network payloads, specifically those generated through our intelligent mutation-based fuzzer model. These payloads were obtained from fuzzing experiments, penetration testing, and network interactions within the SAir-IIoT testbed, capturing both benign and exploitative traffic. The extracted features are analysed to classify vulnerabilities based on likelihood and impact. LLM trained over massive cybersecurity datasets, these datasets include structured vulnerability reports (as found in the National Vulnerability Database (NVD) and the CERT Coordination Center), extracted real-world exploits, CVSS scores, and attack behaviours. In the backend, ensemble strategies and fuzzy logic techniques were applied to enhance the precise capability of risk predictions by correcting classification results on earlier attack data.

Moreover, the model further trained with SAir-IIoT specific datasets of network payloads that are specific to the IoT system, taking into consideration the adaption of the proposal over smart environments. The accuracy of the risk analysis improves as a result of the integration of feature extraction, transformer-based deep learning, and expert-mediated rule-based filtering in the LLM-based approach integrates to enable the accurate assessment and prioritization of the vulnerabilities, consequently advancing the automated vulnerability risk assessment of huge IoT networks. A schematic representation of the functionality in our mutation-based fuzzing framework, where LLM provides instant payload analysis and vulnerability detection during target execution. The LLM extracts 14 main features from network payloads to evaluate their content:

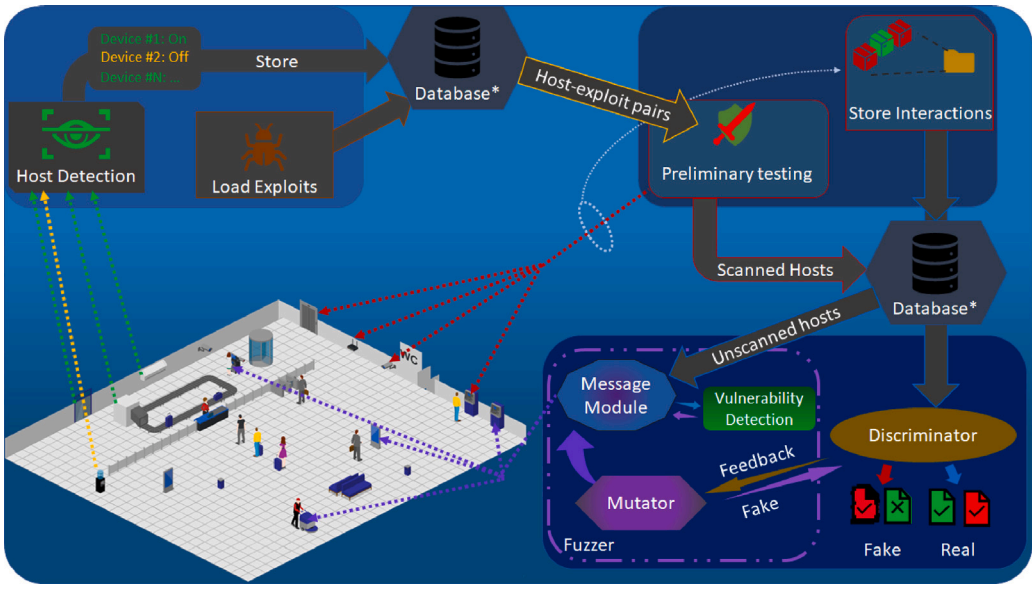


Fig. 2. Example scenario of intelligent mutation fuzzer for IIoT networks.

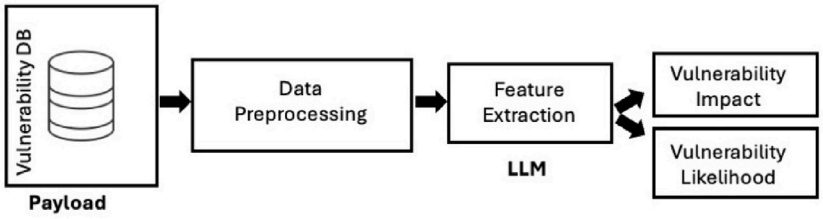


Fig. 3. Architecture of the LLM-driven risk assessment.

- **Protocol Type**: Inferred from the payload if unknown.
- **Payload Size (in bytes)**: Amount of transmitted data.
- **Payload Encoded**: Specifies whether the payload is encoded (e.g., Base64, JSON, or none).
- **Hex Content**: Detects hexadecimal values in the payload.
- **Repetitive Patterns**: Recognizes recurring patterns, which can indicate abnormal or suspicious activity.
- **Authorization Exposure**: Level of access (low, medium, high) to sensitive information.
- **DNS Resolution Failure**: Defines DNS resolution failure, potentially signalling malicious activity.
- **IP Exposed**: Checks if IP addresses are exposed in the payload, which could be a security risk.
- **HTTP Status Code**: Analyzes the HTTP status code (if applicable) to evaluate web interactions.
- **User-Agent and Device Info**: Examines the user-agent string and device information to identify the interaction source.

The LLM uses these features to assess vulnerabilities, applying a five-level scale for **impact** (from very low to very high) and **likelihood** (from very low to very high). The feature extraction process used by the LLM to identify critical attributes of each payload is described in algorithm 7. For example, an SSH payload with many repeating hexadecimal characters would be classified as **high impact** and **high likelihood**, allowing the framework to prioritize and address the most critical vulnerabilities efficiently, ensuring that security threats are properly managed.

In summary, the risk assessment framework operates in two key stages, as illustrated in Fig. 3. First, data preprocessing removes metadata and personally identifiable information (PII) from network payloads collected through fuzzing experiments, penetration testing, and IoT device interactions, ensuring privacy compliance and reducing data noise. Then, feature extraction is performed using the Ollama 3.1 LLM, based on the Llama 3 architecture, to derive 14 key features from the processed payloads. These features are crucial for assessing the payload's characteristics and determining its potential impact and likelihood of exploitation, providing a structured approach to automated vulnerability risk assessment.

Besides adversarial prompt injection, LLMs are also subject to security concerns such as backdoor attacks, prompt injection and training data extraction. Indeed, existing research emphasizes that an adversary can insert a backdoor with hidden nature or design a compromised prompt that defeats the security control system, causing data leakage or undesired functions [46]. To mitigate these risks, our approach ensures no personally identifiable information (PII) is processed, with all metadata removed from fuzzing-generated payloads before analysis. We run Ollama 3.1 locally, preventing external data leakage, and train the LLM exclusively on public cybersecurity datasets (e.g., NVD, CERT). Additionally, we incorporate rule-based filtering and fuzzy logic techniques to enhance reliability and security, ensuring a privacy-conscious risk assessment framework.

4. Experiments and result assessment

In this section, we discuss the experimentation setting that we leveraged to develop and evaluate the proposed Intelligent Network IoT Fuzzer, the so-called DCGAN-MNFuzz. In our previous work, we designed and developed a hybrid experimentation testbed, modelling it to represent a realistic smart airport setting, named SAIr-IIoT testbed [7]. Next, several experiments are presented, illustrating the mutation-based fuzzing capabilities of DCGAN-MNFuzz and their performance in detecting vulnerabilities in smart environment settings. Finally, to

showcase the former's strengths, the proposed mutation-based fuzzer is compared against 3 (could change later) existing state-of-the-art IoT fuzzers.

4.1. Experimentation environment

When developing an AI-enabled cybersecurity solution, acquiring and utilizing high-quality data during training and leveraging a realistic testing environment is imperative. As such, in our previous work, we designed and constructed a hybrid cybertwins testbed for developing cybersecurity solutions for safeguarding smart airports [7]. The testbed comprises three internal and one external zones, and in each zone, several IIoT sensors and actuators, including smoke detectors, IP cameras, pressure, humidity, and occupancy sensors, are deployed and managed by a per-zone coordinator. Each coordinator can bridge communications between devices using a diverse range of wireless communication protocols, such as Bluetooth, ZigBee, Z-Wave, LoRa, WiFi, and Ethernet, ensuring a multi-protocol network representative of real-world deployments.

The UNSW Canberra IoT testbed integrates 82 IIoT devices covering multiple categories, including environmental monitoring sensors, security surveillance cameras, and automation control units. The SAir-IIoT architecture is designed to replicate smart airport infrastructure, ensuring scalability and applicability to real-world IIoT networks. The testbed features a four-tiered architecture:

- Field Tier: Physical IoT sensors deployed in different zones, responsible for collecting real-time telemetry data.
- Communication Tier: Handles data transmission using multiple protocols, ensuring seamless device-to-device and network interactions.
- Edge Tier: Aggregates and processes data using Raspberry Pi gateways with wireless NICs configured for each protocol.
- Cloud & Management Tier: Stores, analyzes, and visualizes data, providing automated pre-processing and real-time security analysis.

The testbed also integrates 20 virtual machines (VMs) to simulate network interactions, allowing a hybrid environment of physical and virtualized IIoT deployments. This approach ensures representativeness by including real hardware alongside simulated IoT devices, effectively mirroring a wide range of industrial and smart infrastructure deployments. SAir-IIoT is designed to facilitate multiple cybersecurity use cases, including experimentation, expert training, device security evaluation, and cybersecurity tool.

By incorporating real-world IoT sensors and communication standards, SAir-IIoT provides a highly representative testing environment for evaluating cybersecurity solutions. These enhancements make it a robust and realistic platform for multi-protocol vulnerability assessments and large-scale IIoT security research.

4.2. Experiments

This experiment has two phases: the first phase is fuzzing-based vulnerability detection, and the second phase is risk assessment for prioritization. To begin with, we designed a mutation-based fuzzing methodology to discover defects in different types of IIoT devices. After this, we used deep learning systems to classify each vulnerability by likelihood and impact level based on the extracted payload features for assessing risk levels. This method aids in the efficient identification and ranking of vulnerabilities in IIoT networks.

4.2.1. Intelligent mutation-based fuzzer

To evaluate the performance of the intelligent mutation-based fuzzer, experiments were conducted on the SAir-IIoT cybertwins testbed, with the fuzzer deployed in a server machine that had network access to the testbed environment. To establish a baseline, the preliminary vulnerability testing module was engaged, which is part of a data management system that was developed as part of the testbed environment. This preliminary vulnerability testing module was designed to leverage Metasploit, a well-established and open-source software used by cybersecurity experts and penetration testing practitioners and is designed to scan the entire network of the testbed environment, including IIoT and non-IIoT devices, for known vulnerabilities. This module follows a multi-process design to reduce the required run-time for scanning every device and open port in the network. Training the intelligent mutation-based fuzzer requires the collection of both benign and malicious network interactions (payloads), the latter of which is obtained by running the preliminary vulnerability testing module against all live devices in the environment.

After the fuzzer has been trained, it can be deployed and leveraged to scan all devices in the network. The fuzzer utilized network interactions obtained by running the preliminary vulnerability testing and the benign interactions collection modules as input, mutating them and forwarding the output to any accessible network-enabled IIoT and non-IIoT devices. Although engaging the preliminary vulnerability testing module is costly (time-wise), requiring over 6 h to complete its execution of the SAir-IIoT environment, the finalized fuzzer was observed to complete its operations in under 30 min. The produced mutations were not only able to detect the previously identified vulnerabilities and verify the performance of the fuzzer but also detected vulnerabilities in the IP cameras that were not detected by the preliminary vulnerability scanning module powered by Metasploit.

4.2.2. Intelligent risk assessment

To improve the risk assessment processing of our intelligent fuzzer, we implemented machine learning models that could classify vulnerabilities by their probability and severity levels. Based on the extracted features of each payload (presented in algorithm 7), we assigned a scale (in terms of likelihood and impact) to each instance using the five-level system consisting of Very Low, Low, Medium, High, and Very High. This classification is crucial for doing vulnerability prioritization properly and identifying and remediating the most potent threats in IIoT networks.

We trained Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs) on labelled data, combining features obtained from the payload with information about vulnerabilities to represent the relationship between the extracted features and likelihood and impact levels. All the models employed learned the underlying relationships using features such as IP exposure, encoded content and repeated patterns to predict accurately its likelihood and impact levels. Using deep learning in this context allows us to dynamically evaluate vulnerabilities in the face of complex feature interactions within real-world IIoT network data. The experimental framework employed deep learning models, systematically fine-tuning layered parameters to optimize the classification of likelihood and impact with maximum accuracy. Table 2 outlines each model's architecture, detailing the number of layers, activation functions, batch size, epochs, and additional configurations.

Each model was compiled with the Adam optimizer and trained over 10 epochs, using a batch size of 32 and categorical cross-entropy as the loss function to classify vulnerabilities into one of five likelihood or impact levels.

4.3. Results and comparison

4.3.1. Results

In our experiments with the intelligent mutation-based fuzzer on the SAir-IIoT cybertwins testbed environment, several mutated probes resulted in responses by both IIoT devices and other connected machines that indicated the presence of a vulnerability. According to the output of the preliminary vulnerability scanning process, as can be seen in Table 3, the intelligent mutation-based fuzzer's performance is assessed.

Table 2
Deep learning model configurations for likelihood and impact classification.

Model	Layers	Units/Filters	Activation function	Pooling	Batch size	Epochs	Optimizer	Loss function
CNN	Conv1D, MaxPooling1D, Dense	64, 128	ReLU	MaxPooling (2)	32	10	Adam	Categorical Crossentropy
RNN	SimpleRNN, Dense	64, 32	ReLU	N/A	32	10	Adam	Categorical Crossentropy
LSTM	LSTM (2 layers), Dense	64, 32	ReLU	N/A	32	10	Adam	Categorical Crossentropy

Table 3
Vulnerabilities and their details explanation.

Vulnerability details	IP	Port	Protocol	Explanation
apple_ios/ssh/cydia _default_ssh	192.168.4.3	22	ssh	The Apple iOS vulnerability "apple_ios/ssh/cydia_default_ssh" poses a threat of remote code execution (RCE) via SSH, allowing attackers to gain unauthorized access to devices. This vulnerability has a CVSS score of 7.5 out of 10.
linux/http/asuswrt _lan_rce	192.168.4.62	443	https	A Linux system running ASUSWRT firmware with an open port 443, potentially vulnerable to remote code execution (RCE) via a LAN connection. This vulnerability is assigned a CVSS score of 9.8/10.
freebsd/http/citrix _dir_traversal _rce	192.168.4.69	443	https	A remote code execution (RCE) vulnerability exists in Citrix products installed on FreeBSD, where an attacker can exploit a directory traversal issue via HTTP requests. CVSS score is 9.8/10.
windows/nimsoft/ nimcontroller _bof	192.168.4.83	139	netbios -ssn	A buffer overflow vulnerability exists in the Nimsoft controller on Windows, which can be exploited by an attacker to execute arbitrary code (CVSS score: 9.0).
linux/http/atutor _filemanager _traversal	192.168.4.83	443	https	An attacker can exploit the atutor file manager traversal vulnerability on Linux systems by sending specially crafted HTTP requests over port 443. CVSS score likely high, around 9 or more.

Vulnerabilities of devices in the SAir-IIoT environment that were found by the fuzzer are displayed in Table 4. Each record consists of the IP address and port number of the device detected as vulnerable, the name of the service running behind each scanned port (if available), a vulnerability description and the mutated payload in UTF-8 format that triggered the vulnerability.

In Table 3, the LLM is an explainable model that turns raw vulnerability data into meaningful suggestions for those in charge of securing information systems. Instead of merely enumerating vulnerabilities, it adds weight and direction by putting them in context (e.g., the apple_ios/ssh/cydia_default_ssh vulnerability, which can lead to RCE when exploited due to default configurations). The LLM is included as part of explainable model, where details about vulnerabilities are parsed to provide a level of importance, supported by elements such as CVSS scores, attack pathways, and likely outcomes. It enables cybersecurity teams in environments comparable to the SAir-IIoT cybertwins testbed to quickly understand why each vulnerability is severe and how significant it will be for their risk identifications, offering greater clarity and relevance. This assists in remediation prioritization.

A comparison of the records of Table 4 with those of Table 3 shows that the developed fuzzer has detected several vulnerabilities that were not detectable by the preliminary vulnerability scanning process. This indicates that the proposed fuzzer can extrapolate from training data and detect novel vulnerabilities by producing GAN-based mutations. Specifically, a vulnerability was detected in the two IP cameras 192.168.4.66 and 192.168.4.69 on ports 80, and 443 respectively. Note that the difference between the preliminary vulnerability scanning process and the intelligent mutation-based fuzzer is that the former relies on established methods of vulnerability detection by employing exploits through Metasploit. At the same time, the latter automatically generates mutations of valid network interactions and assesses the produced responses to find the vulnerabilities.

The accuracy of the three models (CNN, RNN & LSTM) in classifying the Impact and Likelihood levels for a similar vulnerability can be seen in Fig. 4. All models were trained and tested on features with classes labelled available for Impact and Likelihood. As we can see from the results, all of the RNN, CNN, and LSTM models are showing very high accuracies, but still, both RNN and LSTM have performed better than CNN in both classification tasks.

In particular, the CNN model reached scores of 88.81% and 91.46% for Impact classification and Likelihood classification, respectively. For Impact and Likelihood, the improvement on RNN model is 92.10% and 94.04%, respectively. On the other hand, the LSTM model achieved better results — 93.50% for Impact and 94.79% for Likelihood. It implies that using the LSTM model to capture temporal dependencies in the data gives more strength for risk assessment purposes when compared with CNN and RNN in vulnerability-related cyber-attack scenarios across IIoT infrastructure. The False Positive Rate (FPR) and False Negative Rate (FNR) for Impact and Likelihood classifications highlight key misclassification patterns. We observed the highest Impact FNR of 0.0947 in the Medium class, indicating the model struggles the most with false negatives in this category. Similarly, the highest Likelihood FPR of 0.0634 was recorded for the Very High class, suggesting a higher rate of false alarms for likelihood classification in this category. These findings provide critical insights into model performance and its reliability in IIoT vulnerability classification.

A risk matrix plots vulnerabilities across IP addresses and assigns likelihood/impact metrics to calculate the risk level. Within the matrix, vulnerabilities are represented via coloured blocks, where different levels of colouring show the risk level of those vulnerabilities—green indicates lower levels, and red is higher. The rows correspond to the likelihoods (from 'Very Low' to 'Very High'), and the columns represent impact levels on the same scale. This more detailed view from Fig. 5 identifies a vulnerability in the posture and positions and colour intensity, making it easier to spot higher risks. The risk score presented in this illustration is computed for each IP address, as its values are determined by the corresponding descriptors of vulnerability payload (described in Table 4). For example, the IP 192.168.4.66 on port 80 using protocol HTTPS goes into the "Very High" range in likelihood and impact, which combinations for a risk score of 25, shown as a red

Table 4
Intelligent mutation-based fuzzer output.

IP	Port	Vulnerability details	Protocol	Vulnerability payload
192.168.4.34	6667	Unknown	None	:irc.Metasploitable.LAN NOTICE AUTH:*** Looking up your hostname... :irc.Metasploitable.LAN NOTICE AUTH:*** Couldn't resolve your hostname; using your IP address instead
192.168.4.144	1883	Unknown	None	OKairport/0 × 00158d00035a8c13{'gas_density'':0,''linkquality':115,'gas':false}
192.168.4.66	80	Unknown	http	GET /apply.cgi HTTP/1.1 Host: 192.168.4.66 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 12_0_1) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/15.0 Safari/605.1.15 Authorization: Basic YWRtaW46YWRtaW4=
192.168.4.66	80	Unknown	http	HTTP/1.0 503 Service Unavailable\nDate: Fri, 02 Apr 1993 08:32:35 UTC\nContent-type: text/html\nExpires: Thu, 16 Feb 1989 00:00:00 GMT\n\n\x00 503 Service Unavailable\n\n\n\x00
192.168.4.66	80	Unknown	http	pld:'R 1 /AR !Z$ @ @ !9%'' (?vA?!tPR? 0)f44?As$'PtR3@
192.168.4.23	56799	Unknown	None	OZ)airport/zone_02/switches/xiaomi_switch_03{'battery':100, 'voltage':3045,'linkquality':84}
192.168.4.1	80	Unknown	http	HEAD /j2ee/examples/servlets/ HTTP/1.1 Host: _gateway User-Agent: Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html) Connection: close
192.168.4.255	137	Unknown	None	0
192.168.4.69	443	Unknown	https	$pld:'R/R!Z$@ @!9''(?vA?!t@R?0(b44?AqA$PtR2@P<t@U[Q?AO;{}??0X[CeTapt@p[b<=xhYGSS}0 :K!ZZ&@ HzhsYLu)E@ [IZ@?DZa kP$%$?Z1H6D8Mpd4;?(C[Z#Ce=Phm ?Z6 X)5&XnBMl4Ba,@xX,A]/Y$XDxreq@ +?B5=xAH(C!H@?t?L8EkZ 8VA}@<S4vp8?R@@E?j[{}]-$$DEAD%H ?Bp!p@Ae:,?? XRdEqGe$y??#-@GB?@A?2V]A3D?$Xd?GHxXyDYh6H)4l@g?r@DAmE$'A?AXg?4@<'B?{} ÀQmAQ1ZBS6Ÿ'dd8K.$@-IwX&irzZ_4| zZlaX+y h?Vbe'| 81xa? ~ReP?ptYJ@T.| _?ìRRxZS??Z} {}?'resp:\x15\x03\x03\x00 \x02\x02\n
192.168.4.69	443	Unknown	https	$pld:\x1aR\x02\x03\x00\x01\x00\x10\x10\x00\x10\x11\x02\x01\x01\x04\x00\x02/ \x00\x00R \x00\x02\15 !\x00\x02Z\x00\x14\x10\x04\x00\x00\x00\x02\x00\x04\x00\x01\x04 \x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x18(\xd9\xa9\x00\x00\x02vA\xc8 \x9d!\x08 \x00t@R\ x18\xd2\x9a'0(\x1fb\x1b4\x064\xcc

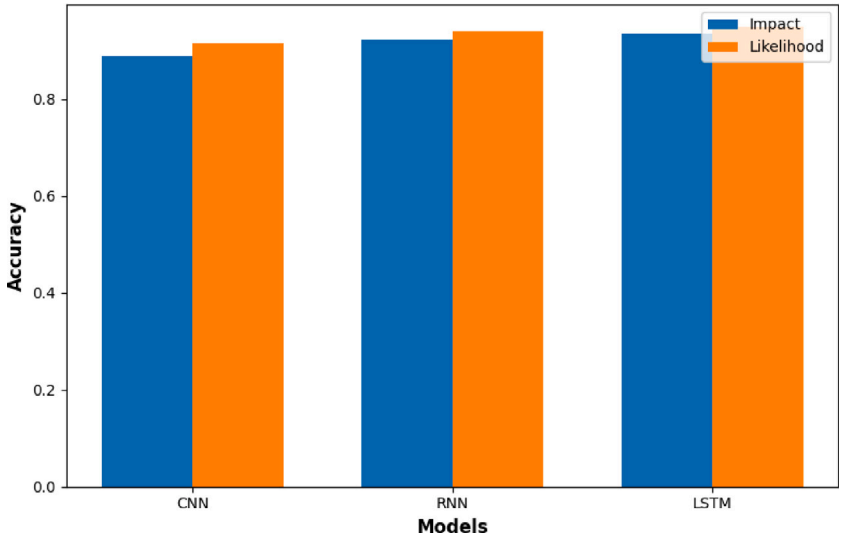


Fig. 4. Model performance in classifying vulnerability impact and likelihood levels.

square in the bottom-right cell of the matrix. This score indicates that an immediate fix and a critical vulnerability are necessary. Likewise, the risk of IP 192.168.4.69 port 443 for the HTTPS protocol is also plotted with a "Medium" likelihood and "High" impact score, indicating a risk level of 12.

Information about vulnerabilities, such as IP addresses, protocols, and payloads, is taken from the analysis in Table 4, which shows the scope of each vulnerability and how critical it is. Therefore, as a visual representation of prioritization, the risk matrix assists security analysts in recognizing high-priority vulnerabilities that should be mitigated immediately in IIoT networks.

4.3.2. Comparison with established IoT fuzzers

The application of fuzzing techniques for the detection of vulnerabilities in computerized systems has been well established since 1990, with a breadth of diverse techniques present in literature that span several domains, including software, device, platform and network fuzzing [47]. However, developing effective fuzzing tools for the IoT and Smart Environment domains remains an open research problem with considerable research activity. Although the same fuzzing principals that are applied in non-IoT settings could be applicable in smart environments, challenges such as non-standard and not-readily available firmware code, the ubiquitous nature of IoT devices in smart environments, and resource restrictions in IoT sensors/actuators assert the need for novel, IoT-specific and efficient fuzzing methods. According to literature [48,49], some of the best IoT fuzzers, with regards to performance and versatility, are: LLMIF [38], LLM-Guided Protocol Fuzzing [39], HFuzz [50] and DIANE [18]. The following is a direct comparison with these tools:

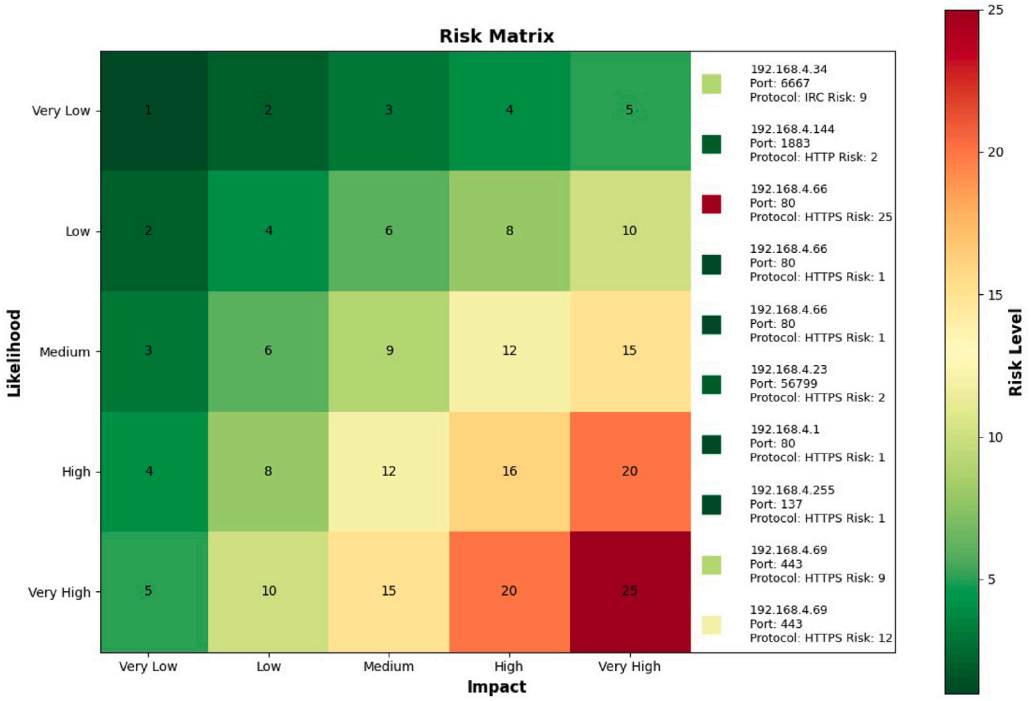


Fig. 5. Risk matrix for identified vulnerabilities.

- LLMIF: It is a protocol-aware fuzzer proposed by Wang et al. [38] to deal with challenges in fuzzing IoT protocols (e.g., Zigbee). In contrast to generic fuzzers, LLMIF can automatically derive protocol specifications (e.g., message payload formats, interesting field values and message dependencies) from reading protocol documents by leveraging a LLM. LLMIF automates this by utilizing such structured protocol information and pairing it to improve each fuzzing phase: seed generation, mutation, and test case enrichment. As per the LLMIF core evaluation, LLMIF outperforms traditional fuzzers in terms of message and code coverage by 55.2% and 53.9%, making it unique among IoT fuzzers. It has also been demonstrated to be very effective in finding vulnerabilities by discovering eight real-world Zigbee devices with unknown vulnerabilities, highlighting its ability to produce non-trivial, syntactically correct payloads that are right on the target for vulnerability detection.
- LLM-Guided Protocol Fuzzing: This fuzzer, developed by Wang et al. [39], aims to fuzz text-based network protocols based on specific semantics. Incorporating an LLM, ProtoGen extracts protocol-specific grammar to support grammar-guided message generation and mutation, addressing challenges unique to text-based protocols. This approach guarantees that mutations still conform to the protocol by preserving structures of messages and dependencies between fields, which is crucial for effective fuzzing. This fuzzer, unlike traditional mutation-based fuzzers, directly generates inputs derived from protocol specifications. Utilizing a LLM to capture protocol-specific grammar enables grammar-guided message generation and mutation, ensuring that mutations adhere to protocol rules and maintain essential message structures and dependencies. This method enhances seed diversity and explores deeper protocol states, leading to substantial improvements in code and state coverage over conventional fuzzers like AFLNET and NSFUZZ and effectively identifying nine previously unknown vulnerabilities across various protocol implementations.
- SGMFuzz: A state-guided mutation-based fuzzer for protocol implementations in IoT contexts [51]. It used a state-guided mutation approach to improve fuzzing by using the information collected during fuzzing to create a finite-state machine (FSM). This FSM was explored further and had complex states and transitions in the program. SGMFuzz added a message-aware module to ensure that the test cases generated by it were well-formed at the protocol level; this way, only the practical protocol parts were fuzzed instead of the entire layer, which wasted resources. In summary, SGMFuzz was designed to maximize the number of execution paths and state transitions that could be explored, thus achieving higher security analysis coverage of protocol implementations. SGMFuzz improved fuzzing efficiency and provided a complete security assessment by reducing invalid inputs and optimizing the mutation process via a state-guided approach. One drawback of SGMFuzz was that it relied on the initial set of seed sequences and the knowledge base from which the fuzzing process was guided. Although this prevented a significant amount of invalid test cases and increased fuzzing efficiency, it was possible that, due to incomplete seed data or a poor knowledge base, the state transitions could not be fully explored.
- HFuzz: Proposed by Liu et al. [50], HFuzz is an IoT mutation-based fuzzer that is designed for applicatframeworkIoT environments and leverages Message Structure Trees (MST) to mutate seed files into fuzz candidates. Primarily evaluated in cellular-based network protocols, HFuzz utilizes information pertaining to protocol structure, thus rendering it a grey-box fuzzer, to achieve greater code coverage. Captured network communications in binary format are initially processed by the MST, resulting in a hierarchical tree representation of captured packets that distinct modules Decomposition and Serialization leverage to perform common bit and byte-wise mutation actions, such as bit flipping, addition, deletion and replacement. This research's primary focus was optimizing the fuzzer, seeking to maximize code coverage while minimizing the required operation time.
- DIANE: Designed by Redini et al. [18], DIANE is an IoT black-box fuzzer that leverages the companion Android/iOS applications to produce fuzzed probes with fewer invalid inputs than existing black-box fuzzing solutions. At its core, DIANE searches for and leverages specific functions found inside the companion applications of IoT devices, the so-called fuzzing triggers to generate

Table 5
Comparison of State-of-the-Art IoT fuzzing tools with our proposed DCGAN-MNFuzz model.

Fuzzer	Fuzzing type	Key features	Advantages	Limitations
LLMIF [38]	Protocol-aware fuzzing	Uses LLM to extract specifications of the protocol (such as message payload format, field values, dependencies etc.) to drive different phases of the fuzzing.	Outperforms traditional fuzzers in message/code coverage by 55.2% and 53.9%. Effective at finding real-world vulnerabilities.	Limited to specific protocols (e.g., Zigbee).
LLM-Guided Protocol Fuzzing [39]	Grammar-guided fuzzing	Uses ProtoGen and LLM to extract protocol-specific grammar for message generation and mutation, ensuring structural integrity.	Ensures mutations preserve protocol rules and structures. Enhanced seed diversity and deeper protocol state exploration.	Limited to text-based protocols.
SGMFuzz [51]	State-guided mutation-based	Combines state-guided mutation with a message-aware module. Uses FSM to guide fuzzing through state transitions, improving resource efficiency by eliminating invalid cases.	Covers most of the execution path/state transitions. Offers Full Security Evaluation.	Dependent on initial seed sequences and knowledge base, may miss some transitions.
HFuzz [50]	Mutation-based fuzzing	Uses Message Structure Trees (MST) to mutate seed files into fuzz candidates. Primarily designed for cellular network protocols with bit/byte-wise mutation.	Focuses on optimizing code coverage while minimizing operation time.	Limited by MST's static protocol structure representation, lacks dynamic exploration.
DIANE [18]	Black-box fuzzing	Leverages companion IoT apps to generate fuzzed probes, using functions inside apps as fuzzing triggers. Detects crashes via response comparison, network packet size, etc.	Reduce Invalid Inputs. Leverages specific functions found in companion apps.	Forced to have companion applications (and thus not very scalable)
DCGAN-MNFuzz (This Model)	Intelligent Mutation-based Fuzzing	Uses GAN to generate mutations based on real network interactions in a smart environment. Integrates an LLM-based Risk Assessment framework to prioritize vulnerabilities.	Utilizes real-world IoT device interactions, enhances fuzzing efficiency, and provides real-time risk prioritization for dynamic environments. Enables more effective payload mutation.	Requires network administration permission and access to encryption keys for payload decryption.

valid fuzzed candidate probes that are correctly structured but have not been modified by the encoding functions found in the application for transmission over the network. Regarding the fuzzing activities that DIANE applies, a similar approach to other such fuzzers is selected, including string length alteration, bit/byte-wise manipulation, and null value insertion, focusing on both primitive variables and more complex objects. DIANE leverages several mechanics to detect crashes, including comparing responses obtained during normal operation, HTTP errors, irregular network packet size and heartbeat monitoring (ping-based). Results indicate that the proposed fuzzer can detect vulnerabilities in IoT devices, however, a similar limitation as with the IoTFuzzer affects versatility(summarized in Table 5).

Although the inspected fuzzers such as LLMIF and LLM-Guided Protocol Fuzzing achieve state-of-the-art results in protocol-aware and grammar-based text protocol fuzzing, their unique features for IoT protocols (e.g., Zigbee) encountered multiple challenges but are still missed by comprehensive risk assessment. These fuzzers then use LLMs to extract/implement protocol structures and message dependencies, significantly augmenting payload mutation and increasing code/message coverage. However, these solutions still need to be improved either by running on a particular protocol or requiring a companion application when executing fuzzing, which negatively impacts its scalability (e.g., HFuzz and DIANE).

On the other hand, Standard commercial static application security testing (SAST) tools like Fortify and Checkmarx [52,53] are among the most prevalent tools for the identification of vulnerabilities during the software development process. These tools examine code (either at the binary or source level) for security defects, such as buffer overflows or injection defects. Nevertheless, they need to reach the codes of the target system, which is frequently inaccessible in IoT and systems because of proprietary firmware and hardware limitations. Moreover, SAST tools are not created for dynamic IoT ecosystems. Our approach operates at the network layer, requiring no prior knowledge of device internals, making it more practical for real-world IoT security.

Dynamic Application Security Testing (DAST) and fuzzing tools (e.g., Burp Suite, Nessus, Peach Fuzzer) target runtime vulnerabilities, deploying payloads against live systems to expose weaknesses and broaden the security attack surface [54,55]. Some fuzzing techniques can catch zero-day vulnerabilities, but traditional DAST relies on preconfigured attack patterns and may not detect entirely new vulnerabilities. By contrast, our DCGAN-MNFuzz approach employs GAN-powered smart fuzzing, where payloads are mutated in real-time, and used to probe application responses to identify vulnerabilities that classic fuzzing and DAST tools can miss.

We proposed an Intelligent Mutation-based Fuzzer for smart environment IoT to address these limitations. An advantage of this fuzzer over the previous works is its ability to utilize network packets that contain interactions among IoT devices and non-IoT devices in a monitored smart environment. The intelligent fuzzer captures valid interactions with real IoT devices and employs a GAN generator trained on benign interactions. It exploits interactions as the mutation model to generate mutated fuzzing data based on recorded device-interaction sequences and sends it to the targeted IoT device. Since this method relies on network captures throughout the entire environment, it can conduct scans against additional varieties of networked devices (the ones that do not require companion smartphone applications for communication) rapidly and at scale, as scanning can be performed towards multiple devices one after another.

What differentiates us is that we include an LLM-based Risk Assessment service within our pipeline, which gives each vulnerability a risk rating based on the likelihood of exposure and impact critical for threat prioritization in IIoT networks. Such a risk evaluation addresses a major

gap in current frameworks, with more ultra-fine-grained details for vulnerability severity than any other fuzzier has provided. Integrating GAN-based fuzzing and risk graph assessment provides an end-to-end, flexible, dynamic solution for large-scale IoT device security in a complex smart environment.

4.3.3. Scalability, generalizability, and Cybertwin-enhanced adaptability

DCGAN-MNFuzz's ability to seamlessly scale across large, complex IIoT environments, as evidenced through its successful evaluation on the sprawling SAir-IIoT testbed, sets it apart from other solutions. This expansive testbed facilitates vulnerability analysis through the integration of no fewer than 82 distinct IIoT devices communicating over an array of protocols including WiFi, Bluetooth, ZigBee, LoRa, Z-Wave and Ethernet. Critically, DCGAN-MNFuzz does not rely on rigid, predetermined attack patterns in the manner of conventional fuzzers but rather leverages its GAN-driven mutations to organically tailor payloads according to the unique attributes of diverse IoT ecosystems independently and without human oversight. Insightfully, a comparative analysis of 8 diverse test infrastructures highlights SAir-IIoT as the most accommodating, garnering a superior scalability score of 125, far surpassing both TON_IoT and IoTBed which achieved ratings of 40 and 11 respectively, validating its preeminent capacity for conducting expansive, real-time security evaluations at scale [7]. Further bolstering its versatility, DCGAN-MNFuzz incorporates a Cybertwins-facilitated architecture to enhance both GAN-powered mutation and risk profiling courtesy of LLMs, allowing it to transition seamlessly between different IoT environments. Collectively, these characteristics entrench DCGAN-MNFuzz as a robust, scalable and accommodating solution suited to address genuine security challenges within large-scale IIoT networks encountered in practice.

When stacked alongside prevailing fuzzers, DCGAN-MNFuzz exhibits clear benefits with respect to scalability. While DIANE augments black-box fuzzing through companion mobile apps, its reliance on application-side validation curtails its potential to scale across diverse IoT ecosystems [18]. LLMIF improves message and code coverage through LLMs assisting protocol inference but remains protocol-specific and primarily designed for Zigbee devices [38]. CHATAFL significantly outperforms AFLNET and NSFUZZ as a protocol fuzzer guided by LLMs, achieving superior state transitions, unique states, and coverage, but optimizes for stateful protocol fuzzing reducing effectiveness for multi-protocol IIoT environments at large scale [39]. Ultimately, with regards to scalability, we illustrate how DCGAN-MNFuzz far outstrips DIANE, LLMIF and CHATAFL in its capacity to smoothly scale across multiple communication protocols, expansive IoT networks and conduct live, dynamic mutation-based fuzzing. These attributes render it a widely generalizable and practical cybersecurity solution for IIoT ecosystems.

5. Conclusion

In this work, we have developed DCGAN-MNFuzz, an intelligent mutation-based fuzzer based on a CNN GAN framework for holistic network-level vulnerability profiling of IoT devices. By using a GAN generator model with a mix of convolutional and dense layers, DCGAN-MNFuzz can generate realistic and protocol-compliant mutations to benign network payloads. Smart environment IoT devices have these mutations running on them to identify vulnerable spots. A second discriminator model, again using convolutional layers, analyses responses from the IoT devices to detect signs of possible weaknesses. Finally, we included an LLM-based analysis tool to help the risk assessments by evaluating the relevance and frequency of identified vulnerabilities. With this component, DCGAN-MNFuzz not only unearths vulnerabilities with high confidence but also organizes them based on their order of mitigation, which is a significant hole in the dynamic risk assessment approach used within IoT networks. In particular, DCGAN-MNFuzz was able to discover device-specific known vulnerabilities that were identified by preliminary scanning as well as previously unknown vulnerabilities across all devices through experimentation over the UNSW Canberra IoT testbed. Combining intelligent fuzzing with LLM-based risk assessment results in a multi-faceted set of capabilities for IoT vulnerability detection and prioritization, meaning DCGAN-MNFuzz can be an effective approach for both advanced systems of static or legacy IoT platforms and supporting the evolving needs of safer smart environments.

CRediT authorship contribution statement

Mohammed Tanvir Masud: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Nickolaos Koroniotis:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis, Data curation. **Marwa Keshk:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis. **Benjamin Turnbull:** Writing – review & editing, Writing – original draft, Visualization, Supervision, Methodology, Data curation. **Shabnam Kasra Kermanshahi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision. **Nour Moustafa:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is funded by the Australian Research Council's Discovery Early Career Researcher Award (DECRA) (Grant Number: DE230100 116), and the is University of New South Wales (UNSW Canberra).

Data availability

The data that has been used is confidential.

References

[1] N. Koroniotis, N. Moustafa, E. Sitnikova, Forensics and deep learning mechanisms for botnets in internet of things: A survey of challenges and solutions, IEEE Access 7 (2019) 61764–61785, http://dx.doi.org/10.1109/ACCESS.2019.2916717.

[2] S. Greengard, The Internet of Things, MIT Press, 2021.

[3] N.M. Karie, N.M. Sahri, P. Haskell-Dowland, IoT threat detection advances, challenges and future directions, in: 2020 Workshop on Emerging Technologies for Security in IoT, ETSecIoT, IEEE, 2020, pp. 22–29.

[4] A.A. Laghari, K. Wu, R.A. Laghari, M. Ali, A.A. Khan, A review and state of art of internet of things (IoT), Arch. Comput. Methods Eng. (2021) 1–19.

[5] N. Koroniotis, N. Moustafa, F. Schiliro, P. Gauravaram, H. Janicke, A holistic review of cybersecurity and reliability perspectives in smart airports, IEEE Access 8 (2020) 209802–209834, http://dx.doi.org/10.1109/ACCESS.2020.3036728.

[6] Y. Hajjaji, W. Boulila, I.R. Farah, I. Romdhani, A. Hussain, Big data and IoT-based applications in smart environments: A systematic review, Comput. Sci. Rev. 39 (2021) 100318.

[7] N. Koroniotis, N. Moustafa, F. Schiliro, P. Gauravaram, H. Janicke, The SAir-IIoT cyber testbed as a service: A novel cybertwins architecture in IIoT-based smart airports, IEEE Trans. Intell. Transp. Syst. (2021) 1–14, http://dx.doi.org/10.1109/TITS.2021.3106378.

[8] D.M. El-Din, A.E. Hassanein, E.E. Hassanien, Smart environments concepts, applications, and challenges, in: Machine Learning and Big Data Analytics Paradigms: Analysis, Applications and Challenges, Springer, 2021, pp. 493–519.

[9] S. Soltan, P. Mittal, H.V. Poor, BlackIoT: IoT botnet of high wattage devices can disrupt the power grid, in: 27th {USENIX} Security Symposium, {USENIX} Security 18, 2018, pp. 15–32.

[10] Y. Shah, S. Sengupta, A survey on classification of cyber-attacks on IoT and iIoT devices, in: 2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference, UEMCON, IEEE, 2020, pp. 0406–0413.

[11] A.A. Süzen, A risk-assessment of cyber attacks and defense strategies in industry 4.0 ecosystem, Int. J. Comput. Netw. Inf. Secur. 12 (1) (2020).

[12] X. Zhang, C. Zhang, X. Li, Z. Du, B. Mao, Y. Li, Y. Zheng, Y. Li, L. Pan, Y. Liu, et al., A survey of protocol fuzzing, ACM Comput. Surv. (2024).

[13] D. Zhang, Q.-G. Wang, G. Feng, Y. Shi, A.V. Vasilakos, A survey on attack detection, estimation and control of industrial cyber–physical systems, ISA Trans. 116 (2021) 1–16.

[14] T. Scharnowski, N. Bars, M. Schloegel, E. Gustafson, M. Muench, G. Vigna, C. Kruegel, T. Holz, A. Abbasi, Fuzzware: Using precise {MMIO} modeling for effective firmware fuzzing, in: 31st USENIX Security Symposium, USENIX Security 22, 2022, pp. 1239–1256.

[15] M.T. Masud, M. Keshk, N. Moustafa, B. Turnbull, W. Susilo, Vulnerability defence using hybrid moving target defence in Internet of Things systems, Comput. Security (2025) 104380.

[16] J. Chen, W. Diao, Q. Zhao, C. Zuo, Z. Lin, X. Wang, W.C. Lau, M. Sun, R. Yang, K. Zhang, IoTFuzzer: Discovering memory corruptions in IoT through app-based fuzzing., in: NDSS, 2018.

[17] D. Wang, X. Zhang, T. Chen, J. Li, Discovering vulnerabilities in COTS IoT devices through blackbox fuzzing web management interface, Secur. Commun. Netw. 2019 (2019).

[18] N. Redini, A. Continella, D. Das, G. De Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, G. Vigna, DIANE: Identifying fuzzing triggers in apps to generate under-constrained inputs for IoT devices, in: 42nd IEEE Symposium on Security and Privacy 2021, 2021.

[19] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, W. Liu, A systematic review of fuzzing techniques, Comput. Secur. 75 (2018) 118–137, http://dx.doi.org/10.1016/j.cose.2018.02.002, URL https://www.sciencedirect.com/science/article/pii/S0167404818300658.

[20] V.J.M. Manès, H. Han, C. Han, S.K. Cha, M. Egele, E.J. Schwartz, M. Woo, The art, science, and engineering of fuzzing: A survey, IEEE Trans. Softw. Eng. (2019).

[21] W. Li, J. Ruan, G. Yi, L. Cheng, X. Luo, H. Cai, {PolyFuzz}: Holistic greybox fuzzing of {multi-language} systems, in: 32nd USENIX Security Symposium, USENIX Security 23, 2023, pp. 1379–1396.

[22] M. Zalewski, American fuzzy lop, 2014, https://lcamtuf.coredump.cx/afl/. (Accessed 10 September 2024).

[23] R. Qian, Q. Zhang, C. Fang, D. Yang, S. Li, B. Li, Z. Chen, DiPri: Distance-based seed prioritization for greybox fuzzing, ACM Trans. Softw. Eng. Methodol. (2024).

[24] Z. Qin, J. Fan, Z. Li, X. Liu, X. Sun, CWGAN-GP: Fuzzing testcase generation method based on conditional generative adversarial network, in: 2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications, TrustCom, 2023, pp. 1286–1293, http://dx.doi.org/10.1109/TrustCom60117.2023.00175.

[25] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, Adv. Neural Inf. Process. Syst. 27 (2014).

[26] M. Mustafa, D. Bard, W. Bhimji, Z. Lukić, R. Al-Rfou, J.M. Kratochvil, CosmoGAN: creating high-fidelity weak lensing convergence maps using generative adversarial networks, Comput. Astrophys. Cosmol. 6 (1) (2019) 1, http://dx.doi.org/10.1186/s40668-019-0029-9.

[27] T.R.V. Bisneto, A.O. de Carvalho Filho, D.M.V. Magalhães, Generative adversarial network and texture features applied to automatic glaucoma detection, Appl. Soft Comput. 90 (2020) 106165.

[28] S.W. Kim, Y. Zhou, J. Philion, A. Torralba, S. Fidler, Learning to simulate dynamic environments with gamegan, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 1231–1240.

[29] I.K. Dutta, B. Ghosh, A. Carlson, M. Totaro, M. Bayoumi, Generative adversarial networks in security: A survey, in: 2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference, UEMCON, IEEE, 2020, pp. 0399–0405.

[30] A. Singh, D. Dutta, A. Saha, MIGAN: malware image synthesis using GANs, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, 2019, pp. 10033–10034, 01.

[31] K. Feng, M.M. Cook, A.K. Marnerides, Sizzler: Sequential fuzzing in ladder diagrams for vulnerability detection and discovery in programmable logic controllers, IEEE Trans. Inf. Forensics Secur. (2023).

[32] H. Yang, Y. Huang, Z. Zhang, F. Li, B.B. Gupta, P. VijayaKumar, A novel generative adversarial network-based fuzzing cases generation method for industrial control system protocols, Comput. Electr. Eng. 117 (2024) 109268.

[33] Z. Wei, X. Wei, X. Zhao, Z. Hu, C. Xu, SGANFuzz: A deep learning-based MQTT fuzzing method using generative adversarial networks, IEEE Access (2024).

[34] Z. Hu, J. Shi, Y. Huang, J. Xiong, X. Bu, GANFuzz: a GAN-based industrial network protocol fuzzing framework, in: Proceedings of the 15th ACM International Conference on Computing Frontiers, 2018, pp. 138–145.

[35] A. Ye, L. Wang, L. Zhao, J. Ke, W. Wang, Q. Liu, RapidFuzz: Accelerating fuzzing via generative adversarial networks, Neurocomputing 460 (2021) 195–204.

[36] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, et al., A survey on evaluation of large language models, ACM Trans. Intell. Syst. Technol. 15 (3) (2024) 1–45.

[37] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y.T. Lee, Y. Li, S. Lundberg, et al., Sparks of artificial general intelligence: Early experiments with gpt-4, 2023, arXiv preprint arXiv:2303.12712.

[38] J. Wang, L. Yu, X. Luo, Llmif: Augmented large language model for fuzzing iot devices, in: 2024 IEEE Symposium on Security and Privacy, SP, IEEE Computer Society, 2024, 196–196.

[39] R. Meng, M. Mirchev, M. Böhme, A. Roychoudhury, Large language model guided protocol fuzzing, in: Proceedings of the 31st Annual Network and Distributed System Security Symposium, NDSS, 2024.

[40] V.-T. Pham, M. Böhme, A. Roychoudhury, AFLNet: a greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification, ICST, IEEE, 2020, pp. 460–465.

[41] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, Nsfuzz: Towards efficient and state-aware network service fuzzing, ACM Trans. Softw. Eng. Methodol. 32 (6) (2023) 1–26.

[42] X. Ma, L. Luo, Q. Zeng, From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter {IoT} devices, in: 33rd USENIX Security Symposium, USENIX Security 24, 2024, pp. 4783–4800.

[43] A. Amro, V. Gkioulos, S. Katsikas, Assessing cyber risk in cyber-physical systems using the ATT&CK framework, ACM Trans. Priv. Secur. 26 (2) (2023) 1–33.

[44] J. Li, B. Zhao, C. Zhang, Fuzzing: a survey, Cybersecurity 1 (1) (2018) 1–13.

[45] Y. Wang, P. Jia, L. Liu, C. Huang, Z. Liu, A systematic review of fuzzing based on machine learning techniques, PLoS One 15 (8) (2020) e0237749.

[46] J. Xu, M.D. Ma, F. Wang, C. Xiao, M. Chen, Instructions as backdoors: Backdoor vulnerabilities of instruction tuning for large language models, 2023, arXiv preprint arXiv:2305.14710.

[47] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, W. Liu, A systematic review of fuzzing techniques, Comput. Secur. 75 (2018) 118–137.

[48] X. Zhu, S. Wen, S. Camtepe, Y. Xiang, Fuzzing: A survey for roadmap, ACM Comput. Surv. (2022).

[49] M. Eceiza, J.L. Flores, M. Iturbe, Fuzzing the internet of things: A review on the techniques and challenges for efficient vulnerability discovery in embedded systems, IEEE Internet Things J. (2021).

[50] X. Liu, B. Cui, J. Fu, J. Ma, HFuzz: Towards automatic fuzzing testing of NB-IoT core network protocols implementations, Future Gener. Comput. Syst. 108 (2020) 390–400, http://dx.doi.org/10.1016/j.future.2019.12.032, URL https://www.sciencedirect.com/science/article/pii/S0167739X19324409.

[51] Z. Wen, J. Yu, Z. Huang, Y. Wu, Z. Hong, R. Ranjan, SGMFuzz: State guided mutation protocol fuzzing, IEEE Netw. Lett. (2025).

[52] OpenText, Fortify static code analyzer, 2024, URL https://www.opentext.com/en-gb/products/fortify-static-code-analyzer. (Accessed 07 February 2025).

[53] Checkmarx, Checkmarx - application security testing, 2025, URL https://checkmarx.com/. (Accessed 07 February 2025).

[54] Peach Tech, Peach fuzzer community edition, 2025, URL https://peachtech.gitlab.io/peach-fuzzer-community/. (Accessed 07 February 2025).

[55] PortSwigger, Burp suite - web security testing, 2025, URL https://portswigger.net/burp. (Accessed 07 February 2025).